

LAB 5. SMART CAMPUS SCENE CLASSIFICATION

MIT INDOOR SCENES 67 WITH VISION TRANSFORMER

ThS. Lê Thị Thùy Trang

2026-04-27

1 Giới thiệu bài thực hành

1.1 Mục tiêu bài thực hành

Sau các bài lab trước, sinh viên đã làm quen với bài toán phân loại ảnh bằng CNN, bài toán chuỗi và cách theo dõi thí nghiệm bằng `Weights & Biases`. Ở bài lab này, chúng ta chuyển sang một kiến trúc hiện đại hơn cho dữ liệu ảnh: **Vision Transformer**, thường viết tắt là **ViT**.

Trong bài lab này, chúng ta không train ViT từ đầu, vì việc đó vừa tốn dữ liệu vừa tốn tài nguyên. Thay vào đó, sinh viên sẽ dùng một mô hình ViT đã được pretrained, sau đó điều chỉnh phần classification head cho bài toán nhận diện ngữ cảnh không gian trong Smart Campus. Nói đơn giản, mô hình sẽ nhìn một ảnh trong môi trường đại học và dự đoán ảnh đó thuộc loại không gian nào.

Cấu hình chính được chọn là **pretrained ViT + head-only training** để lab chạy ổn trên laptop cá nhân. Cấu hình **fine-tune toàn bộ ViT** đã được chuẩn bị trong repo, nhưng được đặt là bài mở rộng cho sinh viên muốn thử nghiệm sâu hơn hoặc có máy đủ mạnh.

- Hiểu bài toán phân loại ngữ cảnh không gian trong bối cảnh Smart Campus;
- Biết cách chuẩn bị subset 5 lớp từ bộ dữ liệu MIT Indoor Scenes 67;
- Hiểu cách ViT biến ảnh thành chuỗi patch để đưa vào Transformer;
- Chạy được mô hình pretrained ViT-B/16 ở chế độ `head_only`;
- Dùng `Weights & Biases` để log quá trình huấn luyện;
- Đọc learning curves và phát hiện overfitting/underfitting;
- Phân tích confusion matrix để biết mô hình hay nhầm lớp nào;
- Rút ra kết luận dựa trên số liệu thay vì cảm giác.

1.2 Kết quả đầu ra mong đợi của bài lab

Sau bài lab này, sinh viên cần có:

1. Một repo chạy được với bài toán phân loại ảnh bằng Vision Transformer;
2. Một subset 5 lớp từ MIT Indoor Scenes 67;
3. Một mô hình ViT-B/16 `head-only` chạy ổn và có log trên W&B;
4. Ít nhất 1 run chính cho baseline ViT `head-only`;
5. Learning curves cho train và validation;

6. Confusion matrix cho kết quả test;
7. File `metrics.json` hoặc bảng tổng hợp metrics;
8. Thống kê `total_params`, `trainable_params` và `trainable_ratio`;
9. Phân tích ngắn: lớp nào dễ bị nhầm, vì sao có thể bị nhầm;
10. Bài mở rộng nếu có: so sánh thêm `head_only` và `finetune`.

1.3 Những thứ bắt buộc phải nộp

Mỗi sinh viên cần nộp tối thiểu:

1. Repo code chạy được.
2. README hướng dẫn chạy.
3. Link hoặc ảnh chụp W&B dashboard.
4. Kết quả của mô hình baseline ViT `head-only`.
5. File `curves.png`.
6. File `confusion_matrix.png`.
7. File `metrics.json` hoặc bảng tổng hợp metrics.
8. Kết quả test của best model.
9. Phân tích ngắn: lớp nào dễ bị nhầm, vì sao có thể bị nhầm.
10. Trả lời phần liên hệ với lý thuyết ViT trong báo cáo.

1.4 Bối cảnh bài toán

Chúng ta đóng vai nhóm kỹ sư AI hỗ trợ xây dựng một module nhỏ trong hệ thống **Smart Campus**. Hệ thống nhận ảnh từ camera hoặc thiết bị quan sát trong trường. Mục tiêu là nhận diện ảnh đó thuộc loại không gian nào, ví dụ phòng học, phòng máy, thư viện, hành lang hoặc văn phòng.

Bộ dữ liệu sử dụng là **MIT Indoor Scenes 67**, sinh viên có thể truy cập đường dẫn [này](#) để tìm hiểu và tải dữ liệu. Trong lab này, chúng ta chỉ dùng subset 5 lớp gần với môi trường đại học:

- classroom
- computerroom
- library
- corridor
- office

Nếu ở Lab 2 chúng ta để CNN nhìn ảnh bằng các bộ lọc tích chập trên không gian ảnh, thì ở Lab 5 này chúng ta để ViT nhìn ảnh như một chuỗi các patch. Điểm thú vị là ViT không bắt đầu bằng convolution như CNN truyền thống. Thay vào đó, ảnh được chia thành các ô nhỏ, mỗi ô được biến thành một vector, rồi toàn bộ chuỗi vector đó được đưa vào Transformer Encoder.

Nói nôm na, với ViT thì ảnh cũng có thể được xem như một câu. Nhưng thay vì câu gồm nhiều từ, ảnh gồm nhiều patch. Mỗi patch giống như một token ảnh.

1.5 Cấu trúc repo starter kit

Repo của bài lab có cấu trúc như sau:

```
csc4005_lab6_mit_indoor_vit_starter/  
  README.md  
  REPORT_TEMPLATE.md  
  requirements.txt  
  configs/  
    baseline_vit_head_only.json  
    debug_smoke.json  
  docs/  
    DATASET_GUIDE.md  
    LAB_GUIDE_LAB5.md  
    RUBRIC.md  
    WANDB_GUIDE.md  
  notebooks/  
    lab6_demo.ipynb  
  outputs/  
  src/  
    __init__.py  
    dataset.py  
    model.py  
    prepare_subset.py  
    train.py  
    utils.py  
  ci/  
    check_structure.py  
    smoke_train.py  
  .github/  
    workflows/  
      ci.yml
```

Ý nghĩa từng phần

- `src/dataset.py`: đọc ảnh trong thư mục subset, resize, normalize và chia train/validation/test.
- `src/model.py`: định nghĩa hàm tạo mô hình ViT-B/16 hoặc ViT-B/32, hỗ trợ `head_only` và `finetune`.
- `src/prepare_subset.py`: tạo subset 5 lớp từ thư mục ảnh gốc của MIT Indoor Scenes 67.
- `src/train.py`: huấn luyện mô hình, đánh giá, lưu output, log W&B.
- `src/utils.py`: các hàm tiện ích như vẽ curves, confusion matrix, lưu metrics.
- `configs/baseline_vit_head_only.json`: cấu hình chính khuyến nghị cho buổi lab.
- `configs/debug_smoke.json`: cấu hình chạy nhanh để kiểm tra pipeline.
- `docs/DATASET_GUIDE.md`: hướng dẫn chuẩn bị dataset.
- `docs/WANDB_GUIDE.md`: hướng dẫn riêng cho W&B.
- `REPORT_TEMPLATE.md`: mẫu báo cáo cho sinh viên.

2 Chuẩn bị môi trường thực hành

2.1 Điều kiện cần

Sinh viên cần:

- Có tài khoản GitHub và đăng nhập được.
- Máy có Internet, đặc biệt trong lần đầu tải pretrained weights.
- Môi trường ảo csc4005-dl đã được set-up từ các buổi trước.
- Đã biết cách dùng wandb login.
- Đã tải được bộ dữ liệu MIT Indoor Scenes 67 hoặc có subset 5 lớp do giảng viên cung cấp.

2.2 Clone repo

Bấm **Accept Assignment** theo link đã cung cấp trên Notion hoặc GitHub Classroom.

Clone repo về máy

```
git clone <repo-url>
cd <ten-repo>
```

2.3 Khởi tạo environment

```
conda activate csc4005-dl
```

Nếu chưa có environment từ các buổi trước thì có thể tạo mới:

```
conda create -n csc4005-dl python=3.10 -y
conda activate csc4005-dl
```

2.4 Cài thư viện phục vụ bài lab

Thực tế thì mình đã cài khá nhiều thư viện từ các bài lab trước rồi, nhưng vẫn để file requirements.txt trong repo và hướng dẫn ở đây cho nó chắc chắn nhé.

```
pip install -r requirements.txt
```

Bài này dùng torchvision để tải pretrained ViT. Nếu máy báo lỗi khi import torchvision, nguyên nhân thường là phiên bản torch và torchvision không tương thích. Đừng vội đổ lỗi cho Transformer, hãy kiểm tra lại môi trường trước nha.

2.5 Chuẩn bị dữ liệu MIT Indoor Scenes 67

Sau khi tải và giải nén, thư mục dữ liệu gốc thường có các thư mục lớp bên trong thư mục ảnh. Ta chỉ cần lấy 5 lớp gần với môi trường đại học:

```
classroom
computerroom
library
corridor
office
```

Có thể dùng script trong repo để tạo subset:

```
python -m src.prepare_subset \
--source_dir /duong_dan/indoorCVPR_09/Images \
--output_dir /duong_dan/mit_indoor_smartcampus_5 \
--classes classroom computerroom library corridor office \
--max_per_class 400
```

Sau khi chuẩn bị, thư mục subset nên có dạng:

```
mit_indoor_smartcampus_5/
classroom/
computerroom/
library/
corridor/
office/
```

Khi chạy, sinh viên truyền đường dẫn tới thư mục này qua tham số `--data_dir`.

Ví dụ:

```
python -m src.train \
--config configs/debug_smoke.json \
--data_dir /duong_dan/mit_indoor_smartcampus_5
```

Nếu dùng Windows, đường dẫn có thể giống như sau:

```
python -m src.train \
--config configs/debug_smoke.json \
--data_dir "D:/datasets/mit_indoor_smartcampus_5"
```

Lưu ý nhỏ nhưng quan trọng: repo không chứa dữ liệu. Không push thư mục `data/`, file zip dataset, file ảnh lớn, file `.pt`, thư mục `outputs/` hoặc thư mục `wandb/` lên GitHub.

2.6 Nhắc lại thao tác với W&B

Nếu máy đã đăng nhập từ buổi trước thì có thể bỏ qua. Nếu chưa, thực hiện lại:

```
wandb login
```

Dán API key khi được yêu cầu.

Trong lab này, W&B là yêu cầu bắt buộc. Project thống nhất của bài lab là:

```
csc4005-lab6-mit-indoor-vit
```

Nếu mạng yếu, có thể chạy không bật `--use_wandb` để debug pipeline trước. Tuy nhiên, run chính thức nộp bài phải có log trên W&B.

3 Voila, hướng dẫn thực hành ở đây

3.1 Kiểm tra pipeline bằng debug smoke

Đừng vội chạy baseline chính ngay. Với ViT, lỗi đường dẫn dữ liệu, lỗi tên folder, lỗi tải pretrained weights, lỗi thiếu RAM hoặc lỗi torchvision đều có thể xảy ra. Vì vậy, ta chạy debug trước cho yên tâm.

```
python -m src.train \
--config configs/debug_smoke.json \
--data_dir /duong_dan/mit_indoor_smartcampus_5
```

Run debug giúp kiểm tra:

- code có tìm được 5 thư mục lớp không;
- ảnh có đọc được bằng PIL không;
- transform resize và normalization có chạy không;
- DataLoader có tạo batch đúng shape không;
- model ViT có forward được không;
- pipeline train/validation/test có chạy hết không;
- output có được lưu vào thư mục outputs/ không.

Nếu debug đã chạy ổn thì mới sang baseline chính. Cứ đi từ nhỏ đến lớn, đỡ đau tim hơn.

3.2 Chạy baseline chính: pretrained ViT head-only

Tinh thần của bài lab này là không train Transformer từ đầu cho thật hoành tráng, mà phải bắt đầu bằng một cấu hình ổn định để hiểu chính xác ViT đang được fine-tune như thế nào.

Ta dùng baseline chính:

```
python -m src.train \
--config configs/baseline_vit_head_only.json \
--data_dir /duong_dan/mit_indoor_smartcampus_5
```

Nếu muốn đặt tên run theo mã sinh viên:

```
python -m src.train \
--config configs/baseline_vit_head_only.json \
--data_dir /duong_dan/mit_indoor_smartcampus_5 \
--run_name mssv_vit_head_only
```

Cấu hình baseline được thiết kế để lab chạy ổn:

- `model_name = vit_b_16`: dùng Vision Transformer base với patch size 16;
- `train_mode = head_only`: chỉ train classification head, giữ nguyên backbone pretrained;
- `img_size = 224`: resize ảnh về kích thước tiêu chuẩn của ViT;
- `batch_size = 16`: cấu hình tương đối vừa phải cho laptop;
- `epochs = 10`: đủ để quan sát xu hướng học;
- `lr = 0.001`: learning rate cho phần head;
- `weight_decay = 0.0001`: regularization L2;
- `dropout = 0.2`: giảm overfitting ở classification head;
- `augment = true`: bật augmentation nhẹ cho train set;
- `use_wandb = true`: bật logging lên W&B.

Nhiều tham số quá nhi, chịu khó đọc giải thích tham số nha.

- `--config`: đường dẫn file cấu hình JSON. Trong lab này nên dùng file config để tránh gõ quá nhiều tham số.

- `--data_dir`: **bắt buộc**, là đường dẫn tới thư mục subset 5 lớp từ MIT Indoor Scenes 67.
- `--project`: tên project trên W&B. Trong lab này dùng thống nhất là `csc4005-lab6-mit-indoor-vit`.
- `--run_name`: tên run.
- `--model_name`: tên mô hình. Với baseline đầu tiên dùng `vit_b_16`.
- `--train_mode`: chế độ train, có thể là `head_only` hoặc `finetune`.
- `--classes`: danh sách lớp cần dùng trong dataset.
- `--img_size`: kích thước ảnh sau khi resize.
- `--dropout`: hệ số dropout trong classification head.
- `--epochs`: số vòng lặp huấn luyện.
- `--batch_size`: số ảnh trong một batch.
- `--lr`: learning rate.
- `--weight_decay`: hệ số regularization L2.
- `--val_ratio`: tỉ lệ validation set.
- `--test_ratio`: tỉ lệ test set.
- `--max_per_class`: giới hạn số ảnh mỗi lớp nếu muốn chạy nhanh.
- `--augment`: bật augmentation nhẹ cho train set.
- `--use_wandb`: bật logging lên W&B.

Chạy baseline xong rồi thì nhớ:

1. Mở dashboard W&B.
2. Kiểm tra train loss và val loss.
3. Kiểm tra train accuracy và val accuracy.
4. Kiểm tra val macro-F1.
5. Kiểm tra số tham số trainable.
6. Mở file `confusion_matrix.png`.
7. Ghi chú lớp nào dễ bị nhầm.
8. Giữ run này làm mốc so sánh với các thử nghiệm mở rộng.

3.3 Vì sao cấu hình chính là head-only chứ không fine-tune toàn bộ?

Đây là phần quan trọng của buổi học. Nếu chỉ chạy code mà không hiểu ý này thì hơi phí.

ViT là mô hình lớn. Nếu train toàn bộ mô hình, sinh viên sẽ cần nhiều tài nguyên hơn, thời gian lâu hơn và dễ gặp lỗi thiếu RAM/GPU hơn. Với một subset nhỏ gồm 5 lớp, việc fine-tune toàn bộ mô hình còn có thể gây overfitting nếu không cẩn thận.

Ở chế độ `head_only`, ta giữ nguyên phần backbone đã được pretrained. Mô hình chỉ học lại phần classification head cho 5 lớp mới. Cách này giống như nói với mô hình: “em đã biết nhìn ảnh tổng quát rồi, bây giờ chỉ cần học cách phân biệt 5 loại không gian trong trường học”.

- `head_only`: train nhanh hơn, ít tốn tài nguyên hơn, phù hợp cho lab.
- `finetune`: linh hoạt hơn, có thể tốt hơn, nhưng tốn tài nguyên và dễ overfitting hơn.

Nói ngắn gọn: trong lab này, ta chọn **ViT head-only** không phải vì fine-tune không hay, mà vì cần một cấu hình đủ ổn định để sinh viên tập trung hiểu đúng quy trình thực nghiệm.

3.4 ViT đang học gì trong bài toán ảnh?

Với ViT, ảnh đầu vào thường có dạng:

```
[batch_size, 3, 224, 224]
```

Nếu dùng patch size 16, ảnh 224 x 224 sẽ được chia thành các patch 16 x 16. Số patch theo mỗi chiều là:

```
224 / 16 = 14
```

Vậy tổng số patch là:

```
14 x 14 = 196 patches
```

Mỗi patch được biến thành một vector, rồi được đưa vào Transformer Encoder. Vì Transformer vốn xử lý chuỗi, ViT coi 196 patch này như một chuỗi token ảnh.

- Patch embedding: biến mỗi patch thành vector.
- Positional embedding: cho mô hình biết patch nằm ở vị trí nào trong ảnh.
- Self-attention: giúp mỗi patch nhìn các patch khác để hiểu toàn cảnh.
- Classification head: biến biểu diễn cuối cùng thành nhãn lớp.

Nếu ở CNN, mô hình học cạnh, góc, texture thông qua convolution, thì ở ViT, mô hình học quan hệ giữa các patch thông qua self-attention.

3.5 Nếu nhìn attention như nhìn tranh trừu tượng thì cũng không sao, đọc hướng dẫn dưới đây là ổn áp ngay

Khi training xong, W&B sẽ cho chúng ta các đường cong học tập.

3.5.1 Những đường quan trọng nhất

- train_loss
- val_loss
- train_acc
- val_acc
- val_macro_f1
- epoch_time_sec

3.5.2 Trường hợp tốt

- train_loss giảm đều,
- val_loss giảm đều hoặc giảm rồi ổn định,
- train_acc tăng,
- val_acc và val_macro_f1 tăng theo,

- khoảng cách giữa train và validation không quá lớn.

Đây là dấu hiệu mô hình đang học đúng hướng.

3.5.3 Dấu hiệu overfitting

- `train_acc` tăng rất cao,
- `train_loss` tiếp tục giảm,
- nhưng `val_loss` bắt đầu tăng,
- `val_acc` hoặc `val_macro_f1` không tăng tương ứng.

Lúc này mô hình đang ghi nhớ train set quá mức. Với subset nhỏ từ MIT Indoor Scenes 67, điều này có thể xảy ra nếu số ảnh mỗi lớp ít, augmentation chưa đủ hoặc fine-tune quá mạnh.

3.5.4 Dấu hiệu underfitting

- cả train accuracy và val accuracy đều thấp,
- train loss và val loss đều còn cao,
- mô hình gần như đoán lung tung sau nhiều epoch.

Lúc này mô hình chưa học đủ hoặc pipeline có vấn đề. Cũng có thể do dữ liệu bị đọc sai, nhãn bị lệch hoặc learning rate chưa phù hợp.

3.6 Đọc confusion matrix

Learning curves cho ta biết mô hình học có ổn không. Confusion matrix cho ta biết mô hình đang nhầm ở đâu.

Sau khi train xong, mở:

```
outputs/<run_name>/confusion_matrix.png
```

Hãy quan sát:

- lớp nào có số dự đoán đúng nhiều nhất;
- lớp nào bị nhầm sang lớp khác nhiều nhất;
- có cặp lớp nào hay nhầm lẫn với nhau không;
- lỗi nhầm đó có hợp lý về mặt hình ảnh không.

Ví dụ, `classroom` và `office` có thể cùng có bàn ghế, màn hình, bảng hoặc tường sáng màu. `corridor` và `library` đôi khi có các đường thẳng, kệ hoặc không gian dài. Mô hình nhầm là chuyện bình thường. Điều quan trọng không phải là giấu lỗi đi, mà là nhìn vào lỗi để hiểu mô hình.

3.7 Thử biến thể fine-tune toàn bộ ViT

Sau khi baseline `head_only` đã chạy ổn, sinh viên có thể thử thêm `finetune`. Đây là bài mở rộng, không bắt buộc với tất cả sinh viên.

```
python -m src.train \
--data_dir /duong_dan/mit_indoor_smartcampus_5 \
--project csc4005-lab6-mit-indoor-vit \
--run_name mssv_vit_finetune \
--model_name vit_b_16 \
--train_mode finetune \
--epochs 5 \
--batch_size 8 \
--img_size 224 \
--lr 0.00005 \
--weight_decay 0.0001 \
--dropout 0.2 \
--augment \
--use_wandb
```

Khi so sánh với head_only, đừng chỉ hỏi “accuracy có cao hơn không?”. Hãy hỏi thêm:

- fine-tune có train chậm hơn không;
- val loss có ổn định hơn không;
- trainable parameters tăng lên bao nhiêu;
- confusion matrix có giảm nhầm lẫn ở lớp nào không;
- kết quả có đáng với chi phí train tăng lên không.

Lưu ý: Không yêu cầu finetune phải tốt hơn head_only. Nếu fine-tune kém hơn, đó vẫn là một kết quả tốt nếu sinh viên giải thích được nguyên nhân.

3.8 So sánh các run trên W&B

Sau khi chạy ít nhất baseline chính, vào project trên W&B và kiểm tra.

Nếu có nhiều run, hãy so sánh:

- run nào có best_val_macro_f1 cao nhất,
- run nào có val_loss thấp nhất,
- run nào learning curve ổn định hơn,
- run nào train nhanh hơn theo epoch,
- run nào ít overfitting hơn,
- run nào có confusion matrix hợp lý hơn,
- run nào có tỉ lệ tham số trainable thấp hơn nhưng vẫn đạt kết quả tốt.

Lưu ý: Kết luận phải dựa trên số liệu và dashboard, không nên viết cảm tính. Không viết kiểu “em thấy model này có vẻ tốt hơn”. Hãy viết “model này có macro-F1 cao hơn, val loss ổn định hơn và nhầm ít hơn ở lớp ...”.

3.9 Bước cuối cùng: so bó đũa, chọn cột cờ

3.9.1 Chọn best model

Best model được chọn dựa trên kết quả train model.

Nguyên tắc đúng là:

1. So sánh các cấu hình trên **validation set**.
2. Chọn **một cấu hình tốt nhất**.
3. Ghi rõ lý do chọn.
4. Chỉ sau đó mới dùng **test set** để đánh giá cuối cùng.

Tiêu chí chọn best config có thể là:

- val macro-F1 cao nhất,
- val accuracy cao,
- val loss thấp và ổn định,
- learning curves đẹp,
- ít overfitting hơn,
- thời gian train/epoch hợp lý hơn,
- confusion matrix dễ giải thích hơn.

Hết sức lưu ý: Không được chọn mô hình chỉ vì train accuracy cao.

3.9.2 Đánh giá trên test set

Sau khi chọn được best config, sinh viên đánh giá trên test set **một lần cuối**.

Trong starter kit, quá trình test đã được thực hiện ở cuối mỗi run. Kết quả được lưu trong:

```
outputs/<run_name>/
```

Thông thường sẽ có:

- best_model.pt
- history.csv
- curves.png
- confusion_matrix.png
- metrics.json
- class_to_idx.json
- config.json

Kết quả nên có:

- test accuracy,
- test macro-F1,
- confusion matrix,
- một vài nhận xét về lớp dễ đúng,
- một vài nhận xét về lớp dễ sai.

4 Khi nào head-only tốt hơn fine-tune?

Đây là câu hỏi quan trọng của lab.

`head_only` thường phù hợp hơn trong buổi lab khi:

- thời gian thực hành ngắn;
- máy sinh viên không quá mạnh;
- dữ liệu không quá lớn;
- muốn một baseline ổn định;
- muốn giảm số tham số cần train;
- mục tiêu chính là hiểu pipeline ViT classification.

Tuy nhiên, điều này không có nghĩa fine-tune là hướng kém. Fine-tune toàn bộ ViT có thể rất mạnh nếu có dữ liệu đủ nhiều, augmentation tốt, learning rate phù hợp và tài nguyên huấn luyện đủ. Trong lab này, fine-tune được đề làm bài mở rộng để sinh viên khá/giỏi thấy được sự khác biệt giữa chỉ train head và cập nhật toàn bộ backbone.

Bởi vậy, kết luận cuối cùng phải dựa trên số liệu của chính các em.

5 Những lỗi rất hay gặp

5.1 Lỗi 1. Sai đường dẫn dữ liệu

Thông báo thường gặp là không tìm thấy thư mục `classroom`, `computerroom`, `library`, `corridor` hoặc `office`. Hãy kiểm tra lại đường dẫn `--data_dir` và cấu trúc folder.

5.2 Lỗi 2. Push dữ liệu hoặc model lên GitHub

Không push file ảnh, file zip dataset, thư mục `data/`, file `.pt`, thư mục `outputs/` hoặc thư mục `wandb/` lên GitHub. Repo chỉ nên chứa code, config và tài liệu.

5.3 Lỗi 3. Nhầm lẫn giữa patch và pixel

Pixel là điểm ảnh riêng lẻ. Patch là một vùng ảnh nhỏ, ví dụ 16 x 16 pixel. ViT không đưa từng pixel riêng lẻ vào Transformer, mà đưa các patch đã được embedding.

5.4 Lỗi 4. Quên positional embedding

Nếu không có positional embedding, Transformer khó biết patch nào ở trên, dưới, trái, phải. Với ảnh, thông tin vị trí rất quan trọng.

5.5 Lỗi 5. Hiểu sai shape của input

Input ảnh đưa vào ViT thường có dạng:

```
[batch_size, 3, 224, 224]
```

Trong đó:

- `batch_size`: số ảnh trong một batch;
- 3: ba kênh màu RGB;
- 224, 224: chiều cao và chiều rộng sau resize.

5.6 Lỗi 6. Batch size quá lớn

ViT nặng hơn CNN nhỏ. Nếu bị lỗi thiếu bộ nhớ, hãy giảm `batch_size`. Ví dụ từ 16 xuống 8, hoặc từ 8 xuống 4.

5.7 Lỗi 7. Chỉ nhìn accuracy mà không nhìn macro-F1

Nếu số lượng ảnh giữa các lớp không cân bằng, accuracy có thể gây hiểu nhầm. Macro-F1 giúp nhìn công bằng hơn giữa các lớp.

5.8 Lỗi 8. Không kiểm tra W&B

Chạy xong mà không có log trên W&B thì rất khó chứng minh quá trình thí nghiệm. Trước khi nộp bài, hãy mở dashboard kiểm tra lại run, config và metrics.

5.9 Lỗi 9. Kỳ vọng fine-tune chắc chắn thắng

Fine-tune nghe có vẻ “xịn” hơn, nhưng không có nghĩa là luôn tốt hơn. Nếu dữ liệu ít hoặc learning rate chưa hợp lý, fine-tune có thể overfit hoặc kém ổn định hơn head-only.

6 Câu hỏi tự kiểm tra sau lab

Hãy tự trả lời ngắn gọn các câu hỏi sau:

1. Vì sao ảnh có thể được xem như một chuỗi patch trong ViT?
2. Patch embedding trong ViT tương tự khái niệm nào trong NLP?
3. Vì sao ViT cần positional embedding?
4. Với ảnh 224 x 224 và patch size 16, có bao nhiêu patch?
5. `head_only` khác gì với `finetune`?
6. Vì sao cấu hình chính dùng `head_only` thay vì fine-tune toàn bộ ViT?
7. Dấu hiệu nào cho thấy mô hình bị overfitting?
8. Confusion matrix giúp phát hiện điều gì?
9. W&B giúp ích gì khi phải so sánh nhiều cấu hình?
10. Nếu fine-tune có kết quả thấp hơn head-only, có thể giải thích như thế nào?

Nếu bạn trả lời rõ ràng được 10 câu này, nghĩa là bạn đã hiểu đúng tinh thần của lab.